PROJECT
DOCUMENTATION

Personal Finance Advisor Bot

An intelligent web assistant for budgeting, expense review and savings planning

Academic programme	SkillWallet – AI Specialist (BFSI)
Project domain	Personal finance and generative AI
Technology stack	Python, Flask, Gemini API, SQLite, HTML, CSS and JavaScript
Prepared by	[Your Name]
Source repository	github.com/alwaysonkar/Personal-Finance-Advisor-Bot

PROJECT REPORT • 2026

Report overview

A guide to the purpose, implementation and evaluation of the application.

The application combines AI-generated guidance with calculations performed by the backend. This separation helps keep financial totals consistent while allowing the language model to provide contextual suggestions.

01	Project background and need
02	Problem definition and aims
03	Technology selection
04	Application design
05	Code organisation
06	End-to-end workflow
07	Data storage and API endpoints
08	Validation and safeguards
09	Interface and user experience
10	Testing and observations
11	Constraints and possible upgrades
12	Summary and references

01 Project background and need

Handling personal money involves more than knowing how much is earned. People also need to understand how their spending is distributed, decide how much to save and make adjustments when a particular expense becomes too large. Maintaining this information manually can be inconvenient, especially when a person wants quick feedback.

The Personal Finance Advisor Bot is a browser-based application designed to make this process easier. A user provides monthly income, expense amounts and a savings objective. The application analyses the information with the Google Gemini model and presents a suggested budget, category-level observations and practical actions.

The project demonstrates a use of generative AI in the banking, financial services and insurance (BFSI) domain. It is intended as a budgeting aid and educational project rather than a replacement for a qualified financial professional.

Core idea: Turn a small set of monthly financial inputs into an understandable spending review and a goal-oriented budget suggestion.

02 Problem definition and aims

Many individuals find it difficult to identify where their monthly income is going. Regular costs such as housing, food and transport can gradually reduce the amount available for savings. Without a simple review process, it may be hard to recognise overspending or measure progress towards a financial target.

This project addresses the need for a lightweight tool that reviews expenses against general budgeting guidelines, produces a proposed allocation and keeps a short history of earlier assessments.

- Provide a straightforward form for entering income, expenses and a savings goal.
- Generate a category-wise review and a suggested budget using a generative AI service.
- Calculate spending totals, remaining balance and savings percentage in Python.
- Keep a record of completed analyses and display recent activity.
- Check user input and respond safely to unavailable services or unexpected AI output.
- Keep backend responsibilities separated to make the application easier to maintain.

03 Technology selection

The implementation uses a small Python web backend, a browser-based interface, a local database and an external generative AI service. Each technology has a defined role in the application.

Technology	Role in the project
Python 3	Implements the application logic and server-side calculations.
Flask 3.0.3	Handles page rendering, application routes and JSON requests.
Google Gemini API (google-genai)	Creates the budget proposal, spending observations and recommendations. The default model is gemini-2.5-flash and can be configured.
SQLite	Stores completed analysis records in a local database file.
python-dotenv	Loads configuration values, including the API key, from the environment file.
HTML, CSS and JavaScript	Build the input form, present results and request data from Flask using fetch.
pyngrok	Can optionally expose a local running instance for demonstration.
Git and GitHub	Support source-code versioning and repository hosting.

The stack keeps deployment requirements modest: SQLite does not require a separate database server, while Flask provides a simple way to connect the frontend to Python logic.

04 Application design

The system follows a client-server arrangement. The browser is responsible for collecting information and displaying the response. Flask receives the request, validates it and coordinates the AI and database modules.

The AI integration and database operations are isolated in separate modules. This avoids placing every responsibility inside a single route handler and makes individual parts easier to inspect.

BROWSER INTERFACE | Form, results and history		
Sends user information to Flask and displays the returned analysis		
↓		
FLASK SERVER | Routes and request handling		
Coordinates validation, AI processing, calculations and storage		
■ ↓ ■		
VALIDATORS	GEMINI SERVICE	DATABASE
Checks submitted values	Prepares prompt and parses response	Saves and reads analysis records
	↓	↓
	Google Gemini API	SQLite database

Figure 1. High-level flow of information through the application.

05 Code organisation

The repository is divided into files according to their responsibilities. The main application file starts Flask, while supporting modules handle configuration, routing, AI communication, validation and persistence.

File or folder	Responsibility
app.py	Creates and starts the Flask application.
config.py	Contains settings, budgeting guidelines and goal descriptions.
routes.py	Defines the home page and the analyse, history, summary and generate endpoints.
gemini_service.py	Creates prompts, communicates with Gemini and extracts JSON from the response.
database.py	Creates the SQLite table and provides save/read operations.
validators.py	Checks the format and range of submitted expense data.
test_app.py	Contains 21 automated tests for key application behaviour.
run_public.py	Provides an optional ngrok-based public demonstration link.
requirements.txt / .env.example	Lists dependencies and shows the required environment settings.
templates/ and static/	Contain the HTML page, stylesheet and browser-side JavaScript.
docs/	Holds project documentation and screenshot resources.

A modular structure reduces coupling. For example, a change to the prompt can be made in the Gemini service without rewriting the database code.

06 End-to-end workflow

A typical budgeting session moves through the following stages:

- **Enter details:** The user supplies monthly income, spending by category and a selected savings goal such as an emergency fund, vacation, gadget purchase or investment.
- **Send request:** Browser-side JavaScript submits the information as JSON to the /analyse endpoint.
- **Validate:** Flask checks the income and expense values before continuing.
- **Prepare context:** The prompt builder adds category amounts, their share of income, the relevant guideline and the selected goal description.
- **Request AI analysis:** The Gemini service asks the model to return a structured JSON response.
- **Read the response:** The application extracts the JSON, including when the response contains Markdown code fences or additional text.
- **Calculate key figures:** Python computes total spending, remaining money and savings rate independently of the model.
- **Save and display:** The summary is written to SQLite and the browser displays the budget, category checks and recommendations.

Budgeting reference values

Spending area	Guideline used in the prompt
Housing / rent	No more than 30% of monthly income
Food	Below 15% of monthly income
Transport	Within 10% of monthly income
Entertainment	Approximately 5–8% of monthly income
Savings	At least 20% of monthly income

07 Data storage and API endpoints

The project uses SQLite and creates a table named **analyses** when the application is first launched. The stored record includes the date and time, monthly income, selected goal and a JSON summary. Individual expense amounts are not retained in the table described by the project.

The summary contains income, total_spent, left_over and savings_rate. Monthly reporting groups saved records by month and calculates totals for the most recent six-month period.

Column	Data type	Meaning
id	INTEGER (primary key)	Unique, automatically increasing record number.
created_at	TEXT	Timestamp of the completed analysis.
income	REAL	Monthly income provided by the user.
goal	TEXT	Savings objective selected for the session.
summary	TEXT (JSON)	Stores the calculated financial summary.

Available application routes

Endpoint	Method	Function
/	GET	Loads the main interface.
/analyse	POST	Validates the form, requests AI analysis, calculates summary values and saves the result.
/history	GET	Returns the five most recent analyses.
/summary	GET	Provides monthly totals covering the latest six months.
/generate	POST	Creates a short promotional post (under 100 words) for a finance product.

08 Validation and safeguards

The backend checks incoming data instead of relying only on browser-side controls. This is important because requests can be sent directly to an endpoint without using the visible form.

Condition	Application response
Income is missing, zero, negative or above the accepted limit	Returns an HTTP 400 validation error.
An expense is not numeric or falls outside the allowed range	Returns an HTTP 400 response identifying the category.
No expense values are provided	Requests at least one expense entry.
Category labels contain unsupported symbols or are too long	Removes symbols and limits the label length to 30 characters.
Gemini API key is unavailable	Returns a configuration error and indicates that the .env file needs attention.
Gemini service cannot be reached	Returns a friendly service error (HTTP 502) and logs technical details.

Condition	Application response
AI output cannot be parsed or required fields are absent	Returns an error asking the user to retry.
Saving to SQLite fails	Logs the failure while allowing the generated analysis to be returned.

Security practices described in the project

- The Gemini API key is kept in the local .env file, which is excluded from version control.
- The .env.example file documents configuration keys without containing secret values.
- SQL statements use placeholders rather than directly joining user input into query strings.
- The local database file is excluded from the Git repository.

09 Interface and user experience

The application uses a single responsive webpage. Its main form is placed near the top so that users can enter their financial information without navigating through several screens.

After submission, the same page presents the suggested budget, an indication of whether each category is within or above its guideline, and a short set of recommended actions. The page also includes a monthly overview and a list of recent analyses.

The browser communicates with the backend using JavaScript fetch requests. This allows the application to update the displayed information without requiring a full page reload.

The source report identifies an input form and result/history areas, but does not provide enough detail to reproduce the exact visual design or verify individual screen elements. Add your own application screenshots here if required by your institute.

Suggested screenshots to insert

- Initial page showing the monthly income, expense and goal fields.
- Example analysis showing the recommended budget and category status.
- Monthly summary and recent analysis history.

10 Testing and observations

The repository includes a test module named test_app.py with 21 automated tests. The tests use Flask's test client. Gemini responses are replaced with fixed test data, and temporary databases are used so that test runs can be repeated without depending on live AI output.

The documented test cases cover the main page, successful analysis, invalid input, summary calculations, history retrieval, promotional text generation and recovery of JSON returned inside Markdown fences.

Test scenario	Expected behaviour	Reported result
Open the home route	Page responds with HTTP 200.	Pass
Submit valid income and expense data	Returns budget, analysis, suggestions and summary.	Pass
Submit zero income	Rejects the request with HTTP 400.	Pass
Enter a non-numeric expense	Returns HTTP 400 and identifies the category.	Pass
Calculate summary for income ■50,000 and spending ■28,000	Remaining balance is ■22,000.	Pass
Request history after one analysis	Returns the saved record.	Pass
Request monthly summary	Returns monthly income, spending and net savings.	Pass
Generate promotional text with product details	Returns the generated post.	Pass
Generate text without product details	Rejects the request with HTTP 400.	Pass
Receive JSON surrounded by Markdown fences	Extracts the JSON successfully.	Pass

The test report records successful outcomes for the listed cases. Actual AI-generated wording can vary according to the API key and model configuration, so the live response is not expected to be identical in every run.

11 Constraints and possible upgrades

The application is a useful demonstration, but the project documentation also identifies areas that could be improved before it is used as a larger or public-facing service.

Current constraints

- The recommendations are generated by an AI model and should not be treated as professional financial advice.
- The application does not include user login; records may be visible to people using the same running instance.
- The system does not independently verify that every AI-proposed budget allocation adds up to the user's income.
- Expense categories are predefined in the current form.

Potential future improvements

- Introduce user accounts and separate each person's analysis history.
- Add a backend validation step to ensure that the suggested allocations do not exceed available income.
- Include charts that make monthly spending patterns easier to understand.
- Allow users to create custom expense categories and choose other currencies.
- Provide PDF or Excel export for saving and sharing a budget review.
- Deploy the application to a cloud platform and provide a stable public URL.

A key improvement would be to validate the AI's proposed budget mathematically before showing it as a final recommendation.

12 Summary and references

The Personal Finance Advisor Bot demonstrates how a web application can combine standard backend programming with a generative AI service to support everyday budgeting. Users enter income, expenses and a savings goal; the application then presents a suggested allocation, category-level feedback and practical next steps.

A useful design decision is that the application calculates important totals in Python rather than relying on the language model for arithmetic. SQLite provides a lightweight way to retain previous summaries, while validation and error handling help the application respond to invalid input and service failures.

Overall, the project provides a foundation for exploring AI-assisted personal finance tools. Its future development can focus on stronger user separation, mathematical checks, richer visual reporting and cloud deployment.

References

- Flask documentation — https://flask.palletsprojects.com
- Google AI / Gemini API documentation — https://ai.google.dev
- SQLite documentation — https://www.sqlite.org/docs.html
- Project source repository — https://github.com/alwaysonkar/Personal-Finance-Advisor-Bot

End of project report